

DEPLOY OR ~~[REDACTED]~~

RSVP link:

<https://gdg.community.dev/events/details/google-gdg-on-campus-kalinga-institute-of-industrial-technology-bhubaneswar-india-presents-deploy-or-die/>



STAND A CHANCE TO WIN
**INTERNSHIP
OPPORTUNITY**
+ EXCLUSIVE GOODIES

- 8TH AUGUST

GEN AI WORKSHOP
09:00 AM - 12:30 PM

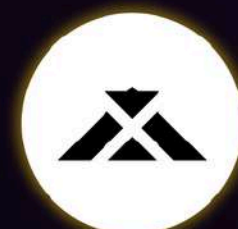
HACKATHON
02:00 PM - 04:00 PM

- 9TH AUGUST

ASTEROID HUNT
09:00 AM - 01:00 PM

C-211, Campus 25

IN COLLABORATION WITH



Deploy or Die: HowToAlgo x GDG on Campus KIIT Hackathon

Our collaborative partner: HowToAlgo



HOWTOALGO

howtoalgo · judge

Tests 6/6 passed

✓ Algorithm Interview Questions

Passed

✓ Machine Learning

Passed

✓ Data Structure Concepts

Passed

✓ System Design Concepts

Passed

✓ System Design Mock Interview

Passed

✓ Behavioral Mock Interview

Passed

🕒 AI Code Review

shipping soon · included free

311+ problems · animated walkthroughs · in-browser judge with live test cases

every future agentic course lands in your account at no extra cost

FOR KIIT · AT THIS EVENT ONLY

One year of everything.

₹2,499 + taxes

₹99

for the full year

YOU SAVE ₹2,400 · 96% OFF

Apply code

GDG99

This price retires the day AI Code Review launches.

SCAN · APPLY GDG99 · YOU'RE IN

howtoalgo.com

by Anuvrittiksha EdTech

Deploy or Die: HowToAlgo x GDG on Campus KIIT Hackathon

THE AGENT-DRIVEN BUILD CHALLENGE

Build a real, working application end to end. You direct AI agents across the full software lifecycle while staying in the loop, and you ship something you can demo, deploy, and defend.

Deploy or Die: HowToAlgo x GDG on Campus KIIT Hackathon

WHAT THIS HACKATHON IS ABOUT

This is not a "write as much code as you can" hackathon. It is an Agent Driven Lifecycle (ADLC) challenge. You will use AI coding agents to take a product from idea to a running, tested, versioned application, and you will be judged on how well you ran that lifecycle, not just on the final screen.

Deploy or Die: HowToAlgo x GDGoC KIIT Hackathon

EVENT FORMAT

Before the event

The problem statements will be released before 8 August. You are free to start planning and building from the moment they are out, so use that time.

Day 1, Build.

You build on campus. Mentors will be available throughout the day to unblock you. Commit continuously to your public repo. Final submissions are taken by the evening of Day 1.

Deploy or Die: HowToAlgo x GDG on Campus KIIT Hackathon



Shortlisting.

Every submitted repo runs through an automated checkpoint script. Repos that clear the gate are scored. The highest scorers become the Top 7 finalists.

Day 2, The Twist (finalists only)

Finalists receive a new requirement that is not announced in advance, to be added to the application they already built. Architect for change from your first commit.

Panel presentation (finalists)

Each finalist team presents to a panel of senior engineers and product leaders. Your presentation is a walk-through of your GitHub repo, a short deck covering problem, approach, architecture, and what the ADLC gave you, and live Q&A on your design decisions and trade-offs.

Deploy or Die: HowToAlgo x GDG on Campus KIIT Hackathon

PROBLEM STATEMENTS

Pick one of three tracks. Each is deliberately broad so any team can find a domain they care about. You define the specific product. We judge how well you build and run it.

- **Track A:** Business Process Automation
- **Track B:** Developer Productivity Tools
- **Track C:** Knowledge and Compliance Agents

TRACK A: BUSINESS PROCESS AUTOMATION

The idea.

Real operational workflows in customer support, finance, HR, and operations are full of repetitive, multi-step, manual work. Pick one real workflow and rebuild it end to end as an agent-driven system, with humans approving the decisions that matter.

Examples:

- A support-ticket triage app that ingests tickets, classifies them, drafts a response, and routes to a human for approval before anything is sent.
- An invoice or expense processing flow that extracts data, checks it against rules, and queues exceptions for a human.
- An employee-onboarding coordinator that generates tasks, tracks completion, and escalates blockers.

TRACK A: BUSINESS PROCESS AUTOMATION

What good looks like.

Every automated step is traceable and auditable. A judge should be able to follow how a decision was made, see where a human approved it, and trust that nothing happened silently.

Best-fit workflow tool.

BMAD, because its full AI product team planning produces the rich PRD and story trail this kind of product benefits from.

TRACK B: DEVELOPER PRODUCTIVITY TOOLS

The idea.

Build a tool that helps software teams themselves. You are students building for developers, so you already understand the pain.

Examples:

- An automated pull-request reviewer that comments on diffs with specific, actionable feedback
- A documentation or changelog generator that reads a codebase and produces readable docs
- A bug-triage assistant that reads incoming issues and suggests labels, severity, and likely owners
- A test-scaffolding tool that generates test cases from code or specs

TRACK B: DEVELOPER PRODUCTIVITY TOOLS

What good looks like.

The tool genuinely fits into a real developer workflow, integrates cleanly as a CLI, GitHub Action, or web app, and is reliable enough that a developer would actually use it.

TRACK C: KNOWLEDGE AND COMPLIANCE AGENTS

The idea.

Build an agent that takes messy, unstructured input such as documents, policies, or meeting transcripts and produces structured, verifiable output: summaries, audits, or answers with citations.

Examples:

- A policy question-answering agent that answers only from provided source documents and cites the exact passage it used
- A compliance-audit agent that checks a document against a set of rules and produces a pass or fail report with evidence
- A transcript-to-action-items extractor that links every action back to where it was said

TRACK C: KNOWLEDGE AND COMPLIANCE AGENTS

What good looks like.

The agent is reliable on messy input and does not hallucinate. Every claim it makes is grounded and verifiable. Citation quality, refusal to invent answers, and graceful handling of bad input all matter here.

Deploy or Die: HowToAlgo x GDG on Campus KIIT Hackathon

THE NON-NEGOTIABLES

Read this slide twice. It is the one that decides whether your work gets looked at.

Every submission runs through an automated checkpoint script first. If the script cannot verify your repo, it goes to manual review against five checkpoints. **If your submission does not clear all five, it does not enter the evaluation pipeline at all.**

No partial credit, no exceptions, no appeals on the morning after.

The five checkpoints:

- **Architecture document:** Present in the repo, describing your stack, data model, and high-level design.
- **Agent rules:** Your rules or constitution file, such as ``.clinerules``, ``.AGENTS.md``, or ``.constitution.md``.
- **Working code:** The app builds and runs. It is demonstrable.
- **At least one custom agent and one custom skill:** Both committed to the repo and documented in ``.AGENTS_AND_SKILLS.md``.
- **A green CI/CD pipeline:** A GitHub Actions workflow is present and the most recent run passes.

These are not scoring criteria. They are entry criteria. A brilliant application with a red pipeline scores zero, because it never reaches a scorer.

Deploy or Die: HowToAlgo x GDG on Campus KIIT Hackathon

WHAT WE ARE LOOKING FOR?

Your work is assessed across four areas.

1. **Specification and Architecture**

Did you decide what you were building, and how, before you started building it? We are looking for deliberate, documented decisions and a clear line from intent to implementation. Reasoning matters more than paperwork.

Deploy or Die: HowToAlgo x GDG on Campus KIIT Hackathon

2. **Working Software and Delivery**

Does it actually run, and can someone else bring it up and see it work? We are looking at the finished application against what you set out to build, and at how automated and repeatable the path from your repository to a running app is.

3. **Agent Engineering and Code Quality**

How deliberately did you shape and steer your AI tooling, rather than prompting a generic assistant? We are looking at how you kept your agent effective as the project grew, and at the state of what came out the other end.

Deploy or Die: HowToAlgo x GDG on Campus KIIT Hackathon

4. Testing and Verification

Is your application verified automatically, and does that verification run as part of your pipeline? This is the smallest band and the one teams most often abandon. Do not. It is also what protects you on Day 2.

Deploy or Die: HowToAlgo x GDG on Campus KIIT Hackathon

HOW YOU WILL BE JUDGED

Step one, the gate.

An automated script checks every repo against the five non-negotiables. Clear the gate and you are scored. Fail it and you are not.

Step two, scoring.

Submissions that clear the gate are evaluated across four areas:

Deploy or Die: HowToAlgo x GDG on Campus KIIT Hackathon

HOW YOU WILL BE JUDGED

Area	Weight
Specification and Architecture	25%
Working Software and Delivery	30%
Agent Engineering and Code Quality	30%
Testing and Verification	15%

Deploy or Die: HowToAlgo x GDG on Campus KIIT Hackathon

HOW YOU WILL BE JUDGED

Step three, finalists.

For the Top 7, the winner is decided on how cleanly you added the Day 2 surprise feature without breaking existing functionality, and on your repo walk-through, deck, and Q&A with the panel.

Deploy or Die: HowToAlgo x GDG on Campus KIIT Hackathon

PRIZES AND SWAGS

For the first 40 registrations.

Event T-shirts, 40 of them, first come first served. Register early.

For everyone who ships.

A co-branded participation certificate from GDGoC and HowToAlgo. Ship something real and it is yours.

Deploy or Die: HowToAlgo x GDG on Campus KIIT Hackathon

PRIZES AND SWAGS

For the top 3 teams.

Gift hampers plus HowToAlgo subscriptions.

For the winning team.

One hour of one-to-one mentoring with senior IT professionals carrying 15 years of industry experience. This is the prize worth optimising for. Hampers get used up. A direct line to people who have shipped software for fifteen years does not.

Deploy or Die: HowToAlgo x GDG on Campus KIIT Hackathon

HOW TO RUN IT, THE STACK

You can use a free, open-source AI coding agent + a free LLM backend + a spec framework.

Editor and agent:

VS Code plus Cline, which is the safest human-in-the-loop option thanks to its Plan and Act approval flow, or Roo Code, which is more autonomous with per-mode models. Both are free and open source.

Deploy or Die: HowToAlgo x GDG on Campus KIIT Hackathon

Spec framework, pick one:

- **GitHub Spec Kit**, the recommended default: lower setup friction, five linear commands, gentle learning curve
- **BMAD-METHOD**: richer artifact trail and a full AI product team, best for Track A, steeper learning curve

AI backend, set up all three so you never stall:

- **NVIDIA Build** at `build.nvidia.com`, your workhorse for the heavy coding loop. Free, no credit card, roughly 1,000 starting credits, strong code models
- **Google AI Studio (Gemini Flash)** for planning, spec generation, and quality-sensitive steps. Permanent free tier, no card
- **A fallback**: OpenRouter, Groq, or a Google Cloud free-trial account

Deploy or Die: HowToAlgo x GDG on Campus KIIT Hackathon

Each team member should create their own keys on each service. Free limits are per account, so a four-person team multiplies its capacity by four.

Deploy or Die: HowToAlgo x GDG on Campus KIIT Hackathon

HOW TO RUN IT, STEP BY STEP

With GitHub Spec Kit, the recommended path:

1. **Install prerequisites:** VS Code, Node.js, Python 3.11+, Git, Docker Desktop, and `uv``
2. **Install the Cline extension.** Under provider settings choose "OpenAI Compatible", paste the NVIDIA base URL `https://integrate.api.nvidia.com/v1`` and your `nvapi-...`` key, then select a code model

Deploy or Die: HowToAlgo x GDG on Campus KIIT Hackathon

3. **Install Spec Kit:** ``uv tool install specify-cli --from git+https://github.com/github/spec-kit.git``, then ``specify init <your-project>``
4. **Run the workflow, letting your agent execute each command while you review every step:** ``/speckit.constitution``, ``/speckit.specify``, ``/speckit.plan``, ``/speckit.tasks``, ``/speckit.analyze`` (always run this read-only consistency check before implementing), then ``/speckit.implement``
5. **Add your custom agent and skill**, CI/CD, Playwright tests, linting, and versioning. Commit continuously and tag a release

Deploy or Die: HowToAlgo x GDG on Campus KIIT Hackathon

With BMAD, best for Track A:

1. Same prerequisites and Cline plus NVIDIA setup
2. Install BMAD: ``npx bmad-method install`` and select your IDE
3. Planning phase, run this in a free web LLM to save your API quota: the Analyst creates a project brief, the PM writes the PRD, the Architect writes the architecture doc
4. Development phase in your IDE: sprint planning, then for each story run draft, develop, test in fresh chats to keep context clean, and finish with a retrospective
5. Add your custom agent and skill, CI/CD, Playwright, linting, versioning. Commit and tag a release

Deploy or Die: HowToAlgo x GDG on Campus KIIT Hackathon

RATE-LIMIT SURVIVAL

Read this one. It will save your event.

- **Split providers by job.** Gemini for planning and hard reasoning, NVIDIA for the bulk implement loop, Groq for fast small edits, OpenRouter as fallback
- **Keep your agent rules tight** so you are not re-sending conventions every task. This is context management and quota management at the same time
- **Use the cheapest capable model** for routine work. Save the strongest model for hard problems

Deploy or Die: HowToAlgo x GDG on Campus KIIT Hackathon

RATE-LIMIT SURVIVAL

- **Request the NVIDIA credit top-up on day one**, before you need it

If a provider starts returning a 429 rate-limit error, switch to your fallback rather than waiting it out

Deploy or Die: HowToAlgo x GDG on Campus KIIT Hackathon

PREREQUISITES, DO THIS BEFORE DAY 1

The teams that set up their tools and accounts in advance win. The teams that start setting up on the morning of the event lose hours they cannot get back.

Accounts to create, all free:

- GitHub account, and verify the GitHub Student Developer Pack with your KIIT email
- NVIDIA Build account at `build.nvidia.com`, no card, then generate an `nvapi-...` key and save it immediately because it is shown only once
- Google AI Studio account, then generate a Gemini API key
- One fallback: OpenRouter or Groq

Deploy or Die: HowToAlgo x GDG on Campus KIIT Hackathon

Tools to install:

- VS Code, Node.js (LTS), Python 3.11+, Git, Docker Desktop, and `uv`
- The Cline or Roo Code VS Code extension, with your NVIDIA key wired in and one successful test message

Verify it works, the day before:

- Send one prompt through Cline using your NVIDIA key and confirm you get a response
- Run `specify init test` or `npx bmad-method install` successfully
- Create a throwaway public GitHub repo and confirm a GitHub Actions workflow runs

Store all API keys in a local `.env` file. Never commit keys to your repo.

Deploy or Die: HowToAlgo x GDG on Campus KIIT Hackathon

RULES AND REQUIREMENTS

- **Team-based.** Team Size: 3-4
- **Your repo must be public** on GitHub. This also gives you unlimited free GitHub Actions minutes
- **No paid Claude Code and no paid GitHub Copilot required.** You can use free, open-source agents with free LLM backends
- **Everything must be demonstrable and validatable.** If a judge cannot run it, see it, or trace it in the repo, it does not count
- **Do not paste secrets, passwords, or personal data** into any AI tool. Free-tier providers may use your inputs to improve their models

Deploy or Die: HowToAlgo x GDG on Campus KIIT Hackathon

RULES AND REQUIREMENTS

- **Human in the loop is required.** Blind, unreviewed auto-generation is not the point and will score poorly
- **Commit continuously.** A single giant end-of-day commit dump scores worse than a clean, progressive history

Deploy or Die: HowToAlgo x GDG on Campus KIIT Hackathon

WHAT TO SUBMIT

End of Day 1, submit your public GitHub repo link.

What that repo contains falls into two groups.

Group one: the five non-negotiables. All five are absolutely necessary.

If any one of these is missing, your submission does not enter the evaluation pipeline at all and nothing else on this slide will save it.

Deploy or Die: HowToAlgo x GDG on Campus KIIT Hackathon

1. **Architecture document.** Present in the repo, describing your stack, data model, and high-level design.
2. **Agent rules.** Your rules or constitution file, such as ``.clinerules``, ``.AGENTS.md``, or ``.constitution.md``.
3. **Working code.** The app builds and runs. It is demonstrable.
4. **At least one custom agent and one custom skill.** Both committed to the repo and documented in ``.AGENTS_AND_SKILLS.md``.
5. **A green CI/CD pipeline.** A GitHub Actions workflow is present and the most recent run passes.

Deploy or Die: HowToAlgo x GDG on Campus KIIT Hackathon

Group two: everything else, in roughly descending order of importance. These do not gate your entry, but they are what your score is built from. Treat them as required, not optional.

6. **Specification or PRD**, with user stories written against acceptance criteria
7. **Playwright end-to-end tests**, passing in CI, with the report uploaded as an artifact
8. **Code-quality configuration.** Your linter or static analysis setup, ideally enforced with pre-commit hooks
9. **A clean, progressive commit history.** A single end-of-day commit dump scores badly
10. **Task breakdown.** The plan your agent worked through
11. **A tagged release.** Semantic versioning and at least one git tag or GitHub Release

Deploy or Die: HowToAlgo x GDG on Campus KIIT Hackathon

Also submit alongside the repo link:

- Confirmation that your CI pipeline is green and your Playwright tests pass
- A short demo video of roughly three minutes, or screenshots of the working app

Finalists, Day 2:

- Updated repo with the new requirement implemented
- Your short presentation deck
- Be ready for the repo walk-through and panel Q&A

Deploy or Die: HowToAlgo x GDG on Campus KIIT Hackathon

QUICK REFERENCE, AI PROVIDER SETUP

Provider	End Point	Key Format	Card Needed	Best Used For
NVIDIA Build	https://integrate.api.nvidia.com/v1	nvapi-...	No	Main coding loop
Google AI Studio	https://generativelanguage.googleapis.com/v1beta/openai/	Gemini API key	No	Planning, quality steps
OpenRouter	https://openrouter.ai/api/v1	sk-or-...	No	Fallback, model variety

Deploy or Die: HowToAlgo x GDG on Campus KIIT Hackathon

QUICK REFERENCE, AI PROVIDER SETUP

Groq	https://api.groq.com/openai/v1	gsk_...	No	Fast small edits
Google Cloud trial	Vertex or Gemini API	project key	Yes	Extra headroom

In Cline or Roo Code, choose the "OpenAI Compatible" provider, paste the endpoint and key, and pick a model. Because BMAD and Spec Kit run inside your editor, they use whatever backend you configured here.

Free AI tiers and their limits change often. Stay updated!

Deploy or Die: HowToAlgo x GDG on Campus KIIT Hackathon

CLOSING

Build something real. Keep your agent on a short leash. Manage your context like it matters, because it does. Make it something you would be proud to demo.